Coding Assignment 2

CS6013 – Advanced Data Structures and Algorithms

Maximum Marks: 90 **Number of Questions: 9**

General Instructions

- Write a separate **C/C++** program for each question, using meaningful variable names and comments for important implementation details.
- Name your source files by question number (e.g., `Q1.cpp`, `Q2.cpp`, or `Q1.c`). Each program must compile and run independently.
- Read from standard input and write to standard output. Follow the specified formats exactly and do not print prompts, debugging messages, or other text.
- Automated evaluation may include hidden and boundary test cases. Do not hard-code sample answers. Implement the algorithm or data structure explicitly required in each question.
- In `README.txt`, provide your name and roll number, compiler version, compilation commands, and instructions for running each program.
- Include any proofs or reports explicitly requested by a question and clearly identify their question numbers.
- Put the source files, `README.txt`, and required reports into a folder named `yourRollNo`. Compress this folder as `yourRollNo.zip` and submit it on Google Classroom. Do not include executables, temporary files, or unrequested datasets.
- Before submission, extract the ZIP archive into a separate folder and verify that every program compiles and runs as documented in `README.txt`.
- This is an individual assignment. Copying others' work is prohibited. Using generative AI tools to generate or modify submitted code or reports is also prohibited. Cite permitted references where applicable; citation does not make otherwise prohibited copying acceptable.
- Submissions may be checked for plagiarism. You may be asked to explain your implementations. Violations will be handled under the institution's academic integrity policy.
- **Each question carries 10 marks.** The submission deadline is on 25th October, 2026 before 11:59 PM.

Question 1: B-Tree Operations

10 Marks

Construct a B-tree by inserting the initial keys in the given order. Each node stores sorted, distinct keys. Every non-root node contains between `MIN` and `MAX` keys, where $\texttt{MAX} = 2\,\texttt{MIN} + 1$. An internal node with k keys has $k + 1$ children, and all leaves are at the same depth. A nonempty root contains between one and `MAX` keys.

Use the following structure or a directly equivalent C++ structure:

```
struct BTreeNode {
    int *key;                    // allocate MAX entries
    int count;
    struct BTreeNode **children; // allocate MAX + 1 entries
};
```

Unused child pointers must be null.

Implement:

(a) `search(root, key)`. (2 marks)

(b) `insert(root, key)`. (3 marks)

(c) `deleteKey(root, key)`. (3 marks)

(d) Level-order display, one tree level per line. (2 marks)

Use the following conventions so that tree displays are reproducible:

- Use top-down insertion. Split a full root before insertion and split a full child before descending into it, promoting the middle key.
- During top-down deletion, before descending into a child with `MIN` keys, borrow from its left sibling if possible, otherwise from its right sibling. A sibling may lend only if it has more than `MIN` keys. If neither can lend, merge with the right sibling if it exists; otherwise merge with the left sibling.
- To delete an internal key, use its predecessor if the left adjacent child has more than `MIN` keys; otherwise use its successor if the right adjacent child does. If both have `MIN` keys, merge them with the separating key and continue deletion.
- Replace an empty internal root with its only child. Represent an empty tree by a null root.
- Duplicate insertions and deletions of absent keys must leave the tree structure unchanged.

Input format

- First line: n, `MAX`, and `MIN`.
- Next n integers: initial keys in insertion order. Repeated initial keys are ignored without producing output.
- Next integer: the number of queries q.
- Each of the next q lines is `1 x` (search), `2 x` (insert), `3 x` (delete), or `4` (display).

Output format For search, print `x is present` or `x is not present`. For insertion, print `x is inserted` or `x is already present`. For deletion, print `x is deleted` or `x is not present. So it can not be deleted`. Replace `x` by the query key.

For display, print each node as comma-separated keys without brackets. Separate nodes on a level by one space, and print levels from top to bottom, with nodes from left to right. For an empty tree, print `EMPTY`. End every display with a separate line containing `END`. Display the tree only for type 4.

Constraints

$$0 \le n \le 10^4, \quad 1 \le q \le 10^4, \quad 1 \le \texttt{MIN} \le 31, \quad \texttt{MAX} = 2\,\texttt{MIN} + 1.$$

Keys lie in $[-10^9, 10^9]$. Across all display queries, the total number of printed keys is at most 2×10^5.

Sample input

```
17 3 1
9 8 5 4 99 78 31 34 89 90 21 23 45 77 88 112 32
8
```

```
1 56
2 21
2 56
2 90
3 51
1 32
3 32
4
```

Sample output

```
56 is not present
21 is already present
56 is inserted
90 is already present
51 is not present. So it can not be deleted
32 is present
32 is deleted
31
8 45,78,90
4,5 9,21,23 34 56,77 88,89 99,112
END
```

Question 2: Red-Black Tree with Order Statistics

10 Marks

Implement a red-black tree supporting:

(a) Insertion while preserving red-black properties. (2.5 marks)

(b) Deletion while preserving red-black properties. (2.5 marks)

(c) Finding the k-th smallest key, with 1-based indexing. (2.5 marks)

(d) Finding the rank of x, defined as the number of stored keys strictly smaller than x. (2.5 marks)

Store each distinct key once. Duplicate insertions and deletions of absent keys leave the tree unchanged. A rank query is valid even when x is absent. Maintain the information required to support every operation in $O(\log(s+1))$ time, where s is the current number of keys. All queries refer to the tree after preceding updates.

Input format

- First line: the number of initial keys n.
- Next n integers: distinct keys in insertion order.
- Next integer: the number of queries q.
- Each of the next q lines contains `type x`: type 1 inserts x, type 2 deletes x, type 3 asks for the x-th smallest key, and type 4 asks for the rank of x.
- For type 3, $1 \le x \le s$ is guaranteed.

Output format Print one line per query:

- Successful insertion: `x is inserted`.
- Duplicate insertion: `x is already present. So no need to insert`.
- Successful deletion: `x is deleted`.
- Absent deletion: `x is not present. So it can not be deleted`.
- Type 3 or 4: print the requested key or rank as a single integer.

Constraints $0 \le n \le 10^5$, $1 \le q \le 10^5$. Initial keys and key arguments lie in $[-10^9, 10^9]$.

Sample input

```
17
9 8 5 4 99 78 31 34 89 90 21 23 45 77 88 112 32
8
1 56
2 51
3 5
4 23
1 21
2 21
3 5
4 22
```

Sample output

```
56 is inserted
51 is not present. So it can not be deleted
21
5
21 is already present. So no need to insert
21 is deleted
23
4
```

Question 3: Range Enumeration in a Red-Black Tree

10 Marks

Construct a red-black tree from the supplied distinct keys. Implement RB-ENUMERATE(x, a, b), where x is the root, to print all keys k satisfying $a \leq k \leq b$. Each range query must take $O(r + \log(n + 1))$ time, where r is the number of keys reported and n is the number of stored keys.

Submit a proof of correctness and a running-time analysis in a separate report labelled **Question 3**.

Input format

- First line: the number of keys n.
- Second line: n distinct keys in insertion order.
- Third line: the number of queries q.
- Each of the next q lines contains two integers a, b, with $a \leq b$.

Output format For each query, print the matching keys in increasing order, separated by spaces, on one line. Print `EMPTY` if no key is in the range.

Constraints $1 \leq n, q \leq 10^5$. Keys and range endpoints lie in $[-10^9, 10^9]$. The total number of keys reported across all queries is at most 2×10^5.

Sample input

```
17
9 8 5 4 99 78 31 34 89 90 21 23 45 77 88 112 32
6
34 88
9 78
23 112
77 99
4 34
113 120
```

Sample output

```
34 45 77 78 88
```

```
9 21 23 31 32 34 45 77 78
23 31 32 34 45 77 78 88 89 90 99 112
77 78 88 89 90 99
4 5 8 9 21 23 31 32 34
EMPTY
```

Question 4: 2-CNF Satisfiability

10 Marks

A 2-CNF formula is a conjunction of clauses, each containing exactly two literals. A literal is a Boolean variable or its negation.

Given a 2-CNF formula on n variables and m clauses, determine whether it is satisfiable. If it is, output any satisfying assignment. Your algorithm must run in polynomial time in n and m.

Input format

- First line: the number of test cases t.
- For each test case, the first line contains n, m.
- Each of the next m lines contains two nonzero signed integers representing a clause. Integer i represents x_i and integer $-i$ represents $\neg x_i$.
- Literal indices satisfy $1 \leq |i| \leq n$. Repeated literals and repeated clauses are allowed.

Output format For each test case, print one line. Print `Unsatisfiable` if no satisfying assignment exists. Otherwise, print n space-separated values, each 0 or 1, in variable order $x_1, \ldots, x_n$, where 0 means false and 1 means true. Any satisfying assignment is accepted.

Constraints $1 \leq t \leq 10^4$, $1 \leq n \leq 2 \times 10^5$, $0 \leq m \leq 2 \times 10^5$. Across all test cases, the sums of n and of m are each at most 2×10^5.

Sample input

```
3
3 3
1 -2
-1 -3
-1 2
2 4
1 2
1 -2
-1 2
-1 -2
2 2
1 2
-1 -2
```

Sample output

```
0 0 1
Unsatisfiable
0 1
```

Question 5: The Forbidden Shortcut

10 Marks

A directed road network has N cities and M one-way roads. Exactly one road has negative travel time; every other road has nonnegative travel time.

Find the minimum total travel time from city S to city T. The negative-weight road may be used at most once. No negative cycle is both reachable from S and able to reach T.

Implement a solution based on Dijkstra's algorithm. The ordinary Dijkstra algorithm must not be applied directly to a graph containing the negative-weight road.

Input format

- First line: N, M, S, T.
- Each of the next M lines contains a_i, b_i, c_i, describing the directed road $a_i \to b_i$ with travel time c_i.
- Exactly one c_i is negative. Parallel roads are allowed.

Output format Print the minimum travel time as one integer. If T is unreachable from S, print `UNREACHABLE`. Use 64-bit signed integers for travel times and path costs.

Constraints

$$2 \le N \le 2 \times 10^5, \quad 1 \le M \le 2 \times 10^5, \quad 1 \le S, T, a_i, b_i \le N.$$

$S \neq T$, $a_i \neq b_i$, and $-10^9 \le c_i \le 10^9$.

Sample input

```
6 7 1 6
1 2 4
2 3 3
3 6 5
1 4 2
4 5 2
5 6 10
2 5 -8
```

Sample output

```
6
```

Question 6: Powering the Cities

10 Marks

There are N cities, none initially electrified. You may build a power station in city i at cost P_i, or build any of the supplied undirected power lines. A city is electrified if it has a station or is connected by built power lines to a city with a station.

Find the minimum total cost to electrify every city. Use a minimum-spanning-tree approach with Prim's or Kruskal's algorithm.

Input format

- First line: N, M, the number of cities and available power lines.
- Second line: $P_1, \ldots, P_N$.
- Each of the next M lines contains u_i, v_i, c_i, describing a possible undirected line with construction cost c_i.
- The graph of available lines is connected. Parallel lines are allowed.

Output format Print the minimum total cost as one integer. Use a 64-bit signed integer for the total.

Constraints

$$1 \le N \le 2 \times 10^5, \quad N - 1 \le M \le 2 \times 10^5, \quad 1 \le P_i \le 10^9, \quad 0 \le c_i \le 10^9.$$

$1 \le u_i, v_i \le N$ and $u_i \ne v_i$.

Sample input

```
4 4
5 2 7 6
1 2 4
2 3 3
3 4 2
1 4 8
```

Sample output

```
11
```

Question 7: Mandatory Roads in a Minimum Spanning Tree

10 Marks

A country has N cities and M possible undirected roads. Road i costs w_i. A specified set of K roads is mandatory.

Find the minimum total cost of a spanning tree containing every mandatory road. If no such tree exists, report that it is impossible.

Implement Kruskal's algorithm with a disjoint-set union structure. Your running time should be $O(M \log(M + 1) + (N + M)\alpha(N))$, where α is the inverse Ackermann function.

Input format

- First line: N, M, K.
- Each of the next M lines contains u_i, v_i, w_i, describing road i between distinct cities u_i, v_i. Roads are numbered from 1 in input order.
- If $K > 0$, the final line contains K distinct mandatory-road indices. If $K = 0$, this line is omitted.

Parallel roads are allowed. The graph need not be connected, and mandatory roads may contain a cycle.

Output format Print the minimum total cost, or `-1` if no spanning tree containing all mandatory roads exists. Use a 64-bit signed integer for the cost.

Constraints

$$1 \le N \le 2 \times 10^5, \quad 0 \le M \le 2 \times 10^5, \quad 0 \le K \le \min(M, N - 1).$$

$1 \le u_i, v_i \le N$, $u_i \ne v_i$, and $0 \le w_i \le 10^9$.

Sample input

```
5 7 2
1 2 4
1 3 2
2 3 1
2 4 5
3 4 3
4 5 2
3 5 6
1 3
```

Sample output

```
10
```

Question 8: Circular Pattern Matching Using KMP

10 Marks

You are given a circular string S of length N and a pattern P of length M, where $1 \le M \le N$. The character after $S[N-1]$ is $S[0]$.

Find all starting positions i, $0 \le i < N$, for which the M consecutive circular characters beginning at i equal P. Include overlapping matches and matches that wrap around the end of S.

Implement the Knuth–Morris–Pratt algorithm, including its prefix-function/LPS array, in $O(N + M)$ time.

Input format

- First line: N, M.
- Second line: S, containing N lowercase English letters.
- Third line: P, containing M lowercase English letters.

Output format Print the number of matches on the first line. If this number is positive, print all 0-based starting positions in increasing order on the second line. Otherwise, print only `0`.

Constraints $1 \le M \le N \le 2 \times 10^5$.

Sample input

```
4 3
abab
bab
```

Sample output

```
2
1 3
```

Question 9: Polynomial Multiplication and the Kronecker Product

10 Marks

(a) Polynomial multiplication using FFT (7 marks)

Given

$$A(x) = \sum_{i=0}^{n-1} a_i x^i, \qquad B(x) = \sum_{j=0}^{m-1} b_j x^j,$$

compute all coefficients of $C(x) = A(x)B(x)$.

Implement multiplication using FFT and its inverse in $O(L \log L)$ time, where L is the smallest power of two with $L \ge n + m - 1$. Direct $O(nm)$ multiplication is not permitted. The output must contain exact integer coefficients; handle floating-point rounding appropriately.

Input format

- First line: n, m, the numbers of coefficients.
- Second line: $a_0, \ldots, a_{n-1}$.
- Third line: $b_0, \ldots, b_{m-1}$.

Output format Print exactly $n + m - 1$ space-separated coefficients $c_0, \ldots, c_{n+m-2}$ in increasing order of degree. Include zero coefficients. Use 64-bit signed integers for the results.

Constraints $1 \le n, m \le 2 \times 10^5$ and $-100 \le a_i, b_j \le 100$. Leading coefficients may be zero.

Sample input

```
3 3
1 2 3
4 5 6
```

Sample output

```
4 13 28 27 18
```

(b) Kronecker product and its connection to FFT (3 marks)

For two n-dimensional vectors $\mathbf{u}$ and $\mathbf{v}$, their Kronecker product $\mathbf{z} = \mathbf{u} \otimes \mathbf{v}$ has entries

$$z_{in+j} = u_i v_j, \qquad 0 \le i, j < n.$$

Consider the claim that FFT can compute and explicitly output $\mathbf{u} \otimes \mathbf{v}$ in $O(n \log n)$ time. Assume each scalar arithmetic operation and output of one scalar takes constant time.

(i) Determine whether the claim is possible and justify your answer.

(ii) Describe an implicit representation using $O(n)$ space that supports querying any entry z_{in+j} in $O(1)$ time.

(iii) Explain the distinction between the Kronecker product and polynomial convolution $s_k = \sum_{i+j=k} u_i v_j$, and explain the role of FFT in the latter.

Submit the program for part (a) as `Q9.cpp` or `Q9.c`. Include the written response for part (b) under a `Q9(b)` heading in `README.txt`. Part (b) has no program input or output.